

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	VIVETHA B	Reg. No.:	192424170
Date:		Duration:	60 minutes

Code of Conduct

I VIVETHA B 192424170 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **VIVETHA B**

Criteria	Mark
Requirement & Pipeline Analysis (15%)	/5
Pipeline Architecture & Workflow (25%)	/4
Tools & Technology Selection (20%)	/4
Automated Testing & Quality Integration (15%)	/4
Deployment, Security & Rollback Strategy (15%)	/4
Pipeline Diagram & Documentation (10%)	/4
TOTAL	/25

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

PHISHING EMAIL DETECTION AND CYBER THREAT INTELLIGENCE SYSTEM

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.

The system is a web-based cybersecurity application that detects phishing emails and provides cyber threat intelligence. It may contain a machine-learning detection model, email/URL analysis, APIs, dashboard, database, and external threat-intelligence services.

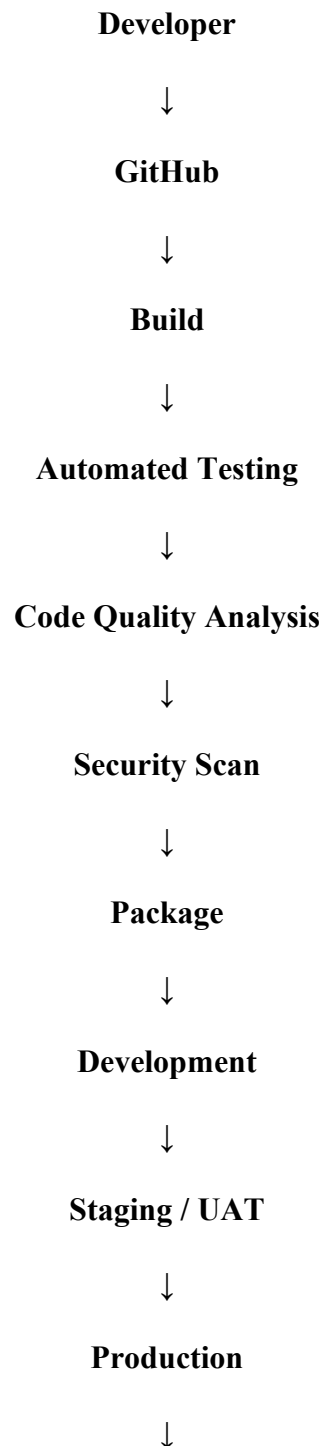
The major CI/CD needs are:

- Frequent updates to the ML model, application code, APIs, and dashboard.
 - Automated building and dependency installation.
 - Automated unit, integration, API, and UI testing.
 - Automated code-quality and security analysis.
 - Containerized application packaging.
 - Automated deployment to different environments.
 - Secure handling of email data, API keys, credentials, and configuration.
 - Continuous monitoring after deployment.
 - Automated notifications about pipeline success or failure.
 - Rollback capability when a deployment fails.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.

The CI/CD pipeline starts when the **Developer** commits code to **GitHub**. The application is then **built** and passed through **automated testing**. Next, **code quality analysis** and **security scanning** are performed. The application is then **packaged** and deployed to the **Development** environment, followed

by **Staging/UAT**. After successful validation and approval, it is deployed to **Production**, where **monitoring and notifications** continuously track system performance and deployment status.

The proposed workflow is:



Monitoring & NotificationsPipeline Stages

Stage 1 – Source Code Management:

Developers commit application code, ML code, configuration files, and tests to GitHub using feature branches and pull requests.

Stage 2 – Build:

Jenkins automatically downloads dependencies, builds the application, and verifies that the project can be successfully compiled or packaged.

Stage 3 – Automated Testing:

Unit, integration, API, UI, and security-related tests are executed automatically. The pipeline stops if critical tests fail.

Stage 4 – Code Quality Analysis:

SonarQube performs static code analysis and checks code coverage, bugs, vulnerabilities, and code smells. A quality gate prevents poor-quality code from continuing.

Stage 5 – Security Scanning:

Dependencies, source code, container images, and secrets are scanned for known vulnerabilities and exposed credentials.

Stage 6 – Packaging:

The application is packaged as a Docker container image and stored in a container registry.

Stage 7 – Deployment:

The Docker image is deployed sequentially to Development, Staging/UAT, and Production environments.

Stage 8 – Monitoring and Notifications:

Prometheus and Grafana monitor application health and performance. Slack or email notifications inform the team about pipeline and deployment status.

3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.

Pipeline Stage	Tool	Purpose	Justification
Source Code	GitHub	Version control	Supports collaboration, branching and pull requests
CI/CD	Jenkins	Pipeline automation	Highly customizable and supports many plugins

Testing	PyTest/JUnit	Unit testing	Suitable for automated application testing
API Testing	Postman/Newman	API validation	Easy automation of API tests
UI Testing	Selenium	UI testing	Automates browser-based testing
Code Quality	SonarQube	Static analysis	Detects bugs, vulnerabilities and code smells
Security	OWASP Dependency-Check	Dependency scanning	Identifies vulnerable dependencies
Security	Trivy	Container scanning	Detects vulnerabilities in Docker images
Packaging	Docker	Containerization	Provides consistent application environments
Registry	Docker Hub	Image storage	Stores versioned Docker images
Deployment	Kubernetes	Container orchestration	Supports scalable and controlled deployments
Monitoring	Prometheus	Metrics collection	Provides continuous system monitoring
Visualization	Grafana	Monitoring dashboard	Provides clear performance visualization
Notification	Slack/Email	Notifications	Provides immediate pipeline alerts

4. Define appropriate **automated testing strategies** to be integrated into the pipeline.

The following automated testing strategies should be integrated into the pipeline:

Unit Testing: Test individual functions such as email preprocessing, URL extraction, feature generation, and phishing prediction.

Integration Testing: Verify communication between the application, database, ML model, and threat-intelligence APIs.

API Testing: Validate REST APIs for email analysis, threat lookup, authentication, and report generation.

UI Testing: Test login, email upload, analysis results, dashboard, alerts, and report-generation functions.

Security Testing: Check for vulnerabilities such as injection attacks, insecure dependencies, exposed secrets, and unauthorized access.

Performance Testing: Test response time and system behavior when multiple users submit emails simultaneously.

ML Model Testing: Test model accuracy, precision, recall, F1-score, false positives, and false negatives using appropriate test datasets.

Regression Testing: Automatically execute previously passed tests whenever new code is introduced to ensure existing functionality is not broken.

5. Design the **deployment strategy**, including development, testing/staging, and production environments.

Deployment Strategy

The deployment strategy for the Phishing Email Detection and Cyber Threat Intelligence System is designed to ensure reliable, secure, and controlled software releases. The application is deployed through multiple environments so that new features, bug fixes, and security updates can be thoroughly tested before reaching end users. This approach reduces deployment risks, improves software quality, and provides a stable production environment.

Development Environment

The Development Environment is the first deployment stage in the CI/CD pipeline. After the source code is committed to the repository and the build process is completed successfully, the application is automatically deployed to the development environment. This environment is mainly used by developers to verify newly implemented features, perform debugging, and test basic functionality.

In this stage, developers validate application components such as email upload, phishing detection, URL analysis, risk-score generation, dashboard functionality, and API integration. Unit testing and basic integration testing are also performed to identify issues early in the software development process. The development environment allows developers to make frequent changes and experiment with new features without affecting the stability of other environments.

The development environment should closely resemble the production setup but can use test databases, sample email datasets, and non-production API keys. Continuous deployment to this environment enables faster feedback and quicker identification of defects.

Testing/Staging Environment

The Testing/Staging Environment acts as an intermediate layer between development and production. Once the application passes development testing and all required quality checks, it is automatically or manually deployed to the staging environment.

The staging environment should closely replicate the production environment in terms of configuration, infrastructure, database structure, APIs, and security settings. This ensures that the application behaves similarly before it is released to actual users.

The following activities are performed in the staging environment:

- Integration Testing
- API Testing
- User Interface Testing
- User Acceptance Testing (UAT)
- Security Testing
- Performance Testing
- Database Validation
- Threat-Intelligence API Verification
- Machine-Learning Model Validation

Quality checks such as code coverage, code quality analysis, dependency scanning, container security scanning, and vulnerability assessment are also verified in this environment. Security policies and authentication mechanisms are tested to ensure that sensitive email and threat-intelligence data remain protected.

The staging environment provides an opportunity for testers, security analysts, and stakeholders to validate the application and confirm that all business requirements have been satisfied before production deployment.

Production Environment

The Production Environment is the final deployment stage where the application becomes available to actual users, administrators, and security analysts. Deployment to production occurs only after successful validation and approval from the staging environment.

Since the system handles cybersecurity-related information, phishing detection, and threat intelligence, production deployment must prioritize reliability, security, and high availability.

To minimize downtime and reduce deployment risks, a rolling deployment strategy can be used. In rolling deployment, application instances are updated gradually rather than updating the entire system at once. This allows users to continue accessing the service while new versions are deployed.

Additional production practices include:

- Continuous monitoring using Prometheus and Grafana.
- Automated alerts for failures and security incidents.
- Centralized logging and audit trails.
- Health checks for application services.
- Backup and disaster recovery mechanisms.
- Automatic rollback to the previous stable version if failures occur.

This approach ensures that the production environment remains stable, secure, and highly available while supporting continuous software delivery.

Deployment Flow

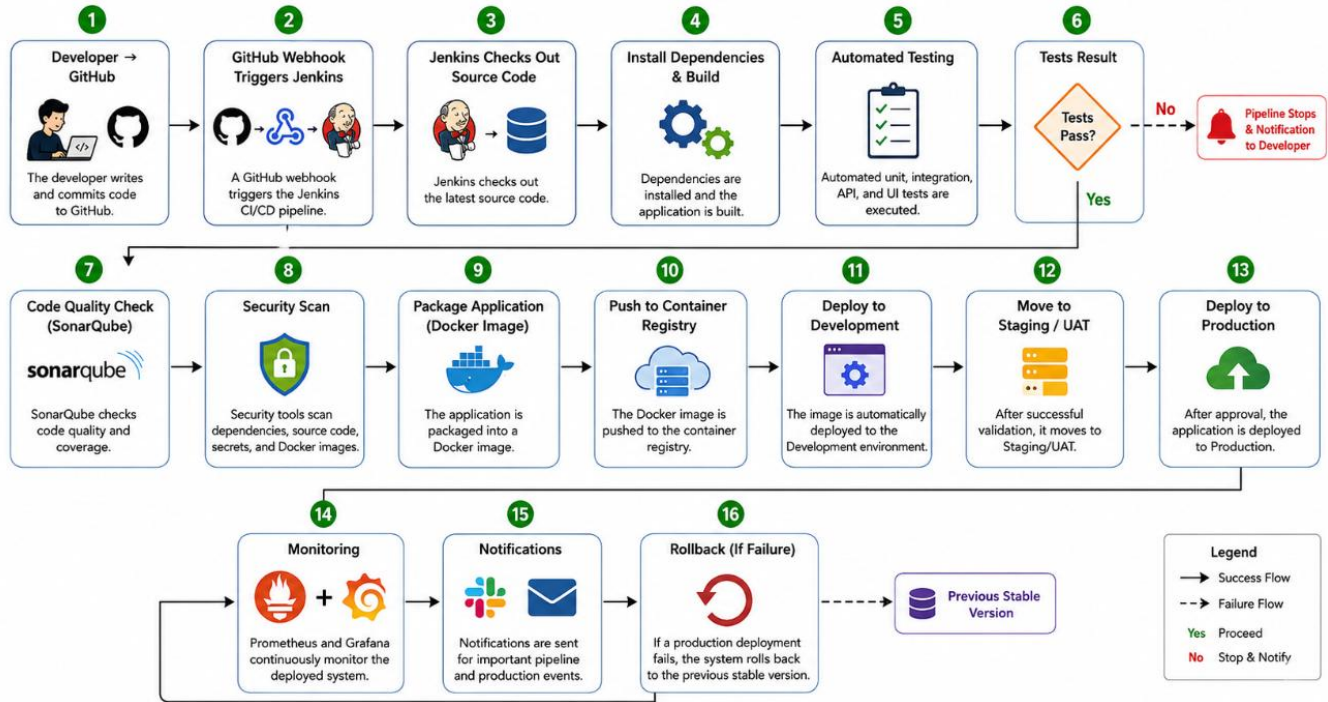
The deployment process follows the sequence shown below:

Development → Testing/Staging (UAT) → Approval → Production

This multi-stage deployment strategy ensures that every code change is tested, validated, secured, and approved before reaching the production environment, thereby improving software quality, reducing risks, and ensuring reliable operation of the Phishing Email Detection and Cyber Threat Intelligence System.

6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.

CI/CD PIPELINE WORKFLOW EXPLANATION



Security

- Use HTTPS for communication.
- Store API keys and passwords as protected secrets.
- Perform dependency vulnerability scanning.
- Scan Docker images using Trivy.
- Perform static security analysis.
- Prevent secrets from being committed to GitHub.
- Apply role-based access control.
- Protect sensitive email and threat-intelligence information.

Quality Checks

- SonarQube quality gate.
- Minimum code coverage threshold.
- Unit and integration test success.
- Security vulnerability checks.
- ML model performance validation.
- Build validation before deployment.

Notifications

The pipeline should send notifications when:

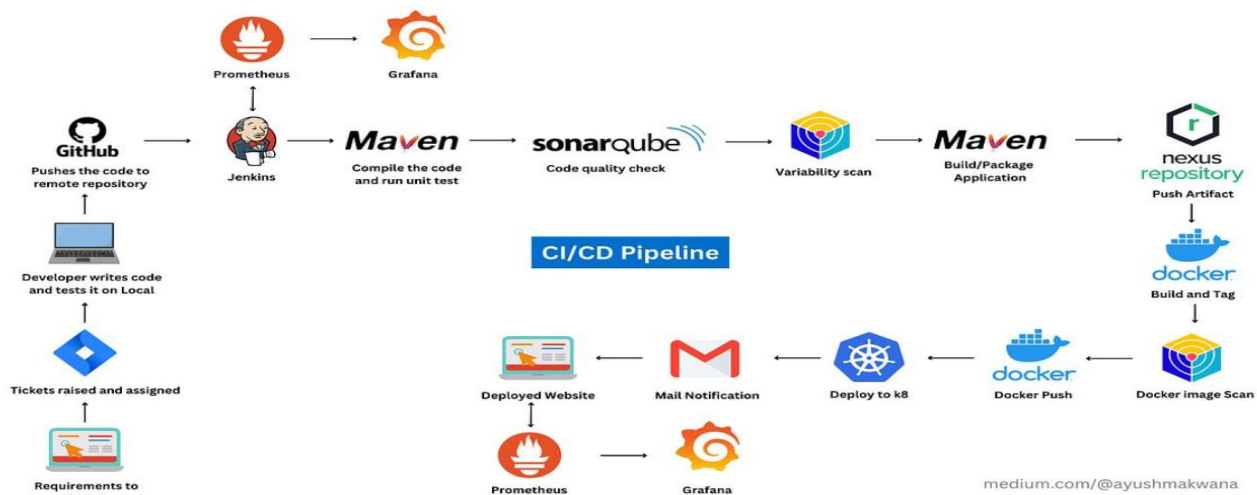
- Build succeeds.
- Build fails.
- Tests fail.

- Security vulnerabilities are detected.
- Deployment succeeds.
- Deployment fails.
- Rollback is triggered.

Rollback

If a production deployment fails, the system should automatically or manually roll back to the **previous stable Docker image/version**. Kubernetes can be used to support controlled rollback.

7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.



Justification

A GitHub-Jenkins based CI/CD architecture is suitable because it automates the complete software delivery process. Automated testing reduces the possibility of defective code reaching production, while SonarQube and security scanners improve code quality and security. Docker ensures consistency between environments, and Kubernetes provides scalable deployment and rollback capabilities. Monitoring and notifications provide continuous visibility into the health of the phishing detection system.

- **GitHub and Jenkins** automate the software development and delivery process, making builds and deployments faster and more consistent.
- **Automated testing** identifies bugs and failures early, reducing the chances of defective code reaching the production environment.
- **SonarQube and security scanners** improve code quality by detecting code issues, vulnerabilities, insecure dependencies, and other security risks.
- **Docker and Kubernetes** provide consistent, scalable, and reliable deployment across Development, Staging, and Production environments.

- **Monitoring and notifications** provide continuous visibility into system performance and quickly alert the team about failures, security issues, or deployment problems.

Conclusion

The proposed CI/CD pipeline provides a **secure, automated, and reliable software delivery process** for the Phishing Email Detection and Cyber Threat Intelligence System. It integrates source control, automated builds, testing, code-quality analysis, security scanning, containerization, deployment, monitoring, notifications, and rollback. This approach enables faster delivery while maintaining the **quality, security, reliability, and availability** of the cybersecurity application.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.